Indoor Navigation with Human Assistance for Service Robots using D*Lite

1st Raulcezar Alves
Center of Information Technology
Federal University of Uberlândia
Uberlândia, Brazil
raul@ufu.br

2nd Josué Silva de Morais
Faculty of Electrical Engineering
Federal University of Uberlândia
Uberlândia, Brazil
josue@ufu.br

3rd Carlos Roberto Lopes
Faculty of Computing
Federal University of Uberlândia
Uberlândia, Brazil
crlopes@ufu.br

***Abstract*—Service robots aim to help humans in their daily tasks at home or office. To accomplish some of these tasks, robots must be able to navigate intelligently from one location to another in dynamic indoor environments. The indoor navigation is a challenging task as many moving and movable objects can be found in the way, blocking the robot's path. D*Lite is a path planning algorithm that replans more efficiently in case the environment changes. However, depending on how blocked the robot is, replanned paths can be much longer than the original or, in the worst case, the target location can not be reached anymore. Therefore, we introduce an adaptation to D*Lite which allows robots to ask for human assistance to move objects on its surrounding when its path is blocked. The algorithm also computes whether and how long the robot can wait for help. Experiments were carried out using simulation and a real robot.**

***Index Terms*—Indoor Navigation, Path Planning, D*Lite, Human Assistance, Service Robots.**

I. Introduction

Although robots already play a very important role in the world today, most of them are used for industrial automation applications. However, many researches aim to develop robots for assisting humans in their everyday tasks.

These robots, also called service robots, should perform tasks that are dirty, dull, distant, dangerous, or repetitive for humans [1]. Many of these tasks, such as vacuum cleaning, delivering mail, patrolling, etc; require the robots to navigate autonomously from one location to another in the environment.

The navigation process aims to plan a path to a target location and to execute this plan while the robot avoids unexpected objects. The navigation in indoor human environments is challenging due to moving and movable objects. In this case, moving objects are dynamic objects able to move fast in the environment, such as people. On the other hand, movable objects are not able to move by themselves, but they can be moved frequently, like furniture.

Tipically, the planning of a path is computed off-line from a static known map of the environment using search algorithms, such as *A** [2] and *Dijkstra* [3]. But when operating in real environments, a mobile robot usually does not have complete information about the map. As a result, any path generated using its initial map may turnout to be invalid or suboptimal as it receives updated map information through onboard sensors, which detect new objects on the way. Therefore, the navigation process must be able to adapt to changes while the robot is executing the path.

To overcome this issue, a new path can be generated from scratch regarding the current location of the robot and the changes found in the map. Another solution is to replan or fix the path more efficiently to go through the environment avoiding the objects detected.

*D** [4] is a dynamic version of *A** which prioritizes computational efficiency rather than optimality in changing environments. It uses heuristics to focus its search and reuses information from previous searches to find solutions to series of similar searches much faster than is possible by solving each search from scratch. *D** and its variants, like its simpler version called *D*Lite* [5], have been widely used for mobile robot and autonomous vehicle navigation including the Mars rovers [6].

Depending on the changes in the environment, the replanned path may be much longer than the original or in the worst case the target location can not be reached anymore.

In this sense, the concept of "Navigation Among Movable Obstacles" (NAMO) [7] proposes that a robot must be able to move objects in its surroundings to reach a previously obstructed location. But this is a very complex task and involves several areas of robotics, like motion, vision, and grasping. Moreover, many popular robotic platforms today still lack arms and hands. Therefore, we introduce an adaptation to the *D*Lite* algorithm which enables the robot to ask for human assistance to move objects on the way based on the replanned path. This can be seen as a symbiotic behavior since robots are performing tasks for humans, and humans are helping robots to complete their tasks.

The main idea is to compare the original path against the replanned one and determine whether the robot needs to ask for human assistance to move objects and how long it can wait for help. While the robot is waiting, it keeps scanning the environment to check if there are changes that allow the replanning of shorter paths.

Although this work is related to the interaction between robots and humans, the goal is to reduce the navigation time by removing obstacles and replannig paths efficiently with search algorithms rather than proposing new Human-Robot Interaction techniques.

2577-1655/18/$31.00 ©2018 IEEE
DOI 10.1109/SMC.2018.00696

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on January 08,2026 at 12:48:25 UTC from IEEE Xplore. Restrictions apply.

The remainder of the paper is organized as follows. Some related works are listed in Section II. Section III presents the proposed approach. A description of the experiments and their results are provided in Section IV. Finally, the conclusion is presented in Section V.

II. Related Work

A number of A^*-like algorithms exist for performing path replanning. *Lifelong Planning A** (*LPA**) [8] is incremental version of A^*. It also uses heuristics to focus the search and always finds a shortest path for the current edge costs. The first search of *LPA** is the same as that of A^* but all subsequent searches are much faster. *LPA** produces at least the search tree that A^* builds. However, it achieves a substantial speedup over A^* because it reuses those parts of the previous search tree that are identical to the new search tree.

The D^* algorithm, a dynamic version of A^*, has been shown to be one to two orders of magnitude more efficient than planning from scratch with A^*, and it has been incorporated into a lot of real robotic systems [9], [10], [11].

*D*Lite* [5] is a hybrid algorithm, taking aspects from both *LPA** and D^*. *D*Lite* has been found to be slightly more efficient by some measures, and it has been used to guide *Segbots* and *ATRV* vehicles in urban terrain [12]. Both D^* and *D*Lite* algorithms guarantee optimal paths over grid-based representations of a robot's environment. Like *LPA**, *D*Lite* replans when it finds divergent values at some arc costs during the navigation, which are classified as overconsistents or underconsistents. There are some differences between *D*Lite* and *LPA**. First, *D*Lite* switches the search direction of *LPA**, i.e., the direction of all edges within the environment are reversed. Another change that *D* Lite* makes to *LPA** is to dynamically move the agent while updating the keys of vertices in its priority queue.

*Delayed D** [13] is a modified version of *D*Lite* that delays the propagation of cost increases as long as possible. By delaying the processing of underconsistent states, *Delayed D** holds two advantages over *D*Lite*. Firstly, it is able to ignore some underconsistent states entirely, reducing the overall computation. Secondly, even when it has to process underconsistent states, if it has delayed processing them long enough then perhaps the effects of various underconsistent states can be incorporated at the same time, in a single propagation.

*Memory-Bounded D*Lite* (*MD*Lite*) [14] is a variant of *D*Lite* that operates in dynamic environments where agents have restricted memory. It uses less memory while maintaing optimality and completeness. *MD*Lite* removes nodes from the priority queue after their expansion. When the algorithm runs out of space and needs to account for additional nodes, it recursively computes parent nodes, adding suitable ones to the queue until all nodes are again accounted for.

*DD*Lite* [15] extends *D*Lite* to support reasoning about state dominance in a domain independent manner. The basic idea underlying search with state dominance is to identify and prune dominated states before they are expanded. It maintains the algorithmic simplicity and incremental search capability of *D*Lite*, whilst enabling orders of magnitude improvements in search efficiency in large state spaces with dominance. In addition, *DD*Lite* is sound, complete, optimal, and efficient.

When planning on a 2D grid, it is common to plan from the center of each grid cell and only allow transitions to the centers of adjacent grid cells. This restricts the agent's heading to increments of $\frac{\pi}{4}$, which results in paths that are suboptimal in length and difficult to traverse in practice. *Field D** [16] and E^* [17] apply interpolation to efficiently produce globally-smooth paths.

Our work is not directly related to search improvements of D^*. Instead, we aim to provide an adaptation that allows robots to ask for human assistance to easily move objects away and find better paths.

This symbiotic behavior can be seen in service robots such as *CoBots* [18]. The *CoBots* can perform different types of tasks, but they have no hands. To overcome this actuation limitation, the robots proactively ask for help from humans, such as asking a human to push the elevator's buttons.

III. D*Lite with Human Assistance

Despite several advances in robotics, robots still have limitations at the perception, cognition, and execution levels. For example, the robot we used in our experiments lacks arms so it could never move blocking objects in its way. However, some of the robots limitations are strengths for humans who coexist with robots in the environment. In this sense, humans can clean robots' paths by lifting and moving objects, while robots are navigating in the environment in order to execute tasks for humans.

Therefore, we introduce an extension to the *D*Lite* algorithm that enables robots to ask for human assistance in case replanned paths, computed to deviate from obstacles, are too long. The algorithm is based on *D*Lite,* using the same notation. The changes required to *D*Lite with Human Assistance* (Algorithm 1) are presented below. Procedures that are the same as presented in *D*Lite* are simply mentioned.

The Main() procedure starts the execution of the algorithm. The variable s_{last} keeps the last initial state used to plan a path to the goal state s_{goal}. Initially, it is set with s_{start} which is the current initial state of the problem (line 27). Then, Main() calls Initialize() procedure to initialize the search problem (line 28).

After that, Main() executes ComputeShortestPath(), which contains the main loop of the algorithm (line 29). This procedure retrieves the shortest path as it is done in *D*Lite*.

Then, the robot starts executing the path until it reaches s_{goal} (lines 30-56). At each iteration of the execution, the robot selects the best successor of the current state (line 32) and moves to that state (line 33). Meanwhile, changes in the environment might occur. Therefore, onboad sensors scan the environment to detect modifications (line 34). If it is detected any change on edge costs of the map (line 35), the algorithm updates the parameter k_m (line 36), and sets again s_{last} as been the current state s_{start} (line 37). The parameter k_m is used to calculate states keys and prioritize the *OPEN* list U.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on January 08,2026 at 12:48:25 UTC from IEEE Xplore. Restrictions apply.

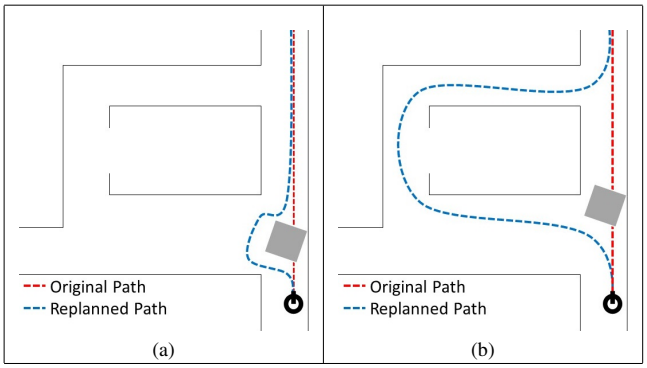

Fig. 1: Replanning Deviations in Obstructed Paths: a) small deviation, b) large deviation.

For all edges with changed costs, the procedure UpdateVertex() is called to update states potentially affected by the changes as well as their membership in the priority queue U (lines 38-40). The variable p_{size} receives the size of the current path (line 41), calculated by the procedure CalculatePathSize() (lines 21-26) which computes the path from the current state to the goal state and returns its size.

Once again, the algorithm calls ComputeShortestPath() to replan the path given the changes detected (line 42). The variable p'_{size} receives the size of the path after replanning (line 43). The algorithm proceeds by comparing the size of both paths, original and replanned (line 44).

If the difference is less than the predefined threshold *min_deviation*, the robot executes the replanned path as the deviation can be done quickly. This scenario is depicted by Figure 1 (a). There, the robot is positioned at the bottom right of the image. The original path, dashed in red, is blockled by a gray box. The replanned path, dashed in blue, shows the deviation necessary to avoid the object. As this is a small deviation, the robot decides to perform the replanned path rather than waiting for human assistance.

Otherwise the robot sends messages, by playing an audio file and also printing a text on its console, to ask for assistance in case the replanned path is much longer the original (line 45), as shown in Figure 1 (b). The gray box is blocking the corridor where the robot goes, which requires a deviation to the other side. However, it can not wait indefinitely because maybe there is nobody around to help. Thus, we compute a timeout based on the difference between both paths and the average speed of the robot (line 46). During this time, if anyone comes to clean at least part of the original path, the robot can reach the destination faster than if it takes the replanned path.

While it waits (lines 47-56), the robot keeps scanning the environemnt. If any change is detected (line 49) the robot replans again, which means that someone probably has cleaned the way. Thus, the algorithm proceeds as before (lines 50-55) and breaks the timeout loop (line 56).

Algorithm 1 D*Lite with Human Assistance

procedure CalculateKey(s)
1: return $[min(g(s), rhs(s)) + h(s_{start}, s) + k_m; min(g(s), rhs(s))]$;

procedure Initialize()
2: $U = \emptyset$;
3: $k_m = 0$;
4: for all $s \in S$ $rhs(s) = g(s) = \infty$;
5: $rhs(s_{goal}) = 0$;
6: U.insert(s_{goal},CalculateKey(s_{goal}));

prodecure UpdateVertex(u)
7: if $(u \neq s_{goal})$ $rhs(u) = min_{s' \in Succ(u)}(c(u,s') + g(s'))$;
8: if $(u \in U)$ U.remove(u);
9: if $(g(u) \neq rhs(u))$ U.insert(u,CalculateKey(u));

prodecure ComputeShortestPath()
10: while (U.TopKey() <CalculateKey(s_{start}) OR $rhs(s_{start} \neq g(s_{start}))$
11: $k_{old} = U$.TopKey();
12: $u = U$.Pop();
13: if (k_{old} <CalculateKey(u))
14: U.insert(u,CalculateKey(u));
15: else if $(g(u) > rhs(u))$
16: $g(u) = rhs(u)$;
17: for all $s \in Pred(u)$ UpdateVertex(s);
18: else
19: $g(u) = \infty$;
20: for all $s \in Pred(u) \cup \{u\}$ UpdateVertex(s);

procedure CalculatePathSize()
21: $P = \emptyset$;
22: $s = s_{start}$;
23: while $(s \neq s_{goal})$
24: P.insert(s);
25: $s = min_{s' \in Succ(s)}(c(s,s') + g(s'))$;
26: return $|P| * cell_size$ /* in centimeters */;

procedure Main()
27: $s_{last} = s_{start}$;
28: Initialize();
29: ComputeShortestPath();
30: while $(s_{start} \neq s_{goal})$
31: /* if $(g(s_{start}) = \infty)$ then there is no known path */
32: $s_{start} = arg\ min_{s' \in Succ(s_{start})}(c(s_{start}, s') + g(s'))$;
33: Move to s_{start};
34: Scan graph for changed edge costs;
35: if any edge costs changed
36: $k_m = k_m + h(s_{last}, s_{start})$;
37: $s_{last} = s_{start}$;
38: for all directed edges (u,v) with changed edge costs
39: Update the edge cost $c(u,v)$;
40: UpdateVertex(u);
41: p_{size} = CalculatePathSize();
42: ComputeShortestPath();
43: p'_{size} = CalculatePathSize();
44: if $(|p'_{size} - p_{size}| > min_deviation)$
45: Ask for Help;
46: $timeout = |p'_{size} - p_{size}| * average_speed$;
47: while $(timeout > 0)$
48: Scan graph for changed edge costs;
49: if any edge costs changed
50: $k_m = k_m + h(s_{last}, s_{start})$;
51: $s_{last} = s_{start}$;
52: for all directed edges (u,v) with changed edge costs
53: Update the edge cost $c(u,v)$;
54: UpdateVertex(u);
55: ComputeShortestPath();
56: break;

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on January 08,2026 at 12:48:25 UTC from IEEE Xplore. Restrictions apply.

IV. Experiments

The navigation experiments were carried out using virtual and real environments. These experiments aim to ensure that we kept the properties of *D*Lite* for *D*Lite with Human Assistance*, and to evaluate the impact of human assistance during the navigation in indoor environments.

The first experiment aims to analyze the efficiency of *D*Lite with Human Assistance* for replanning when the original path is blocked. Therefore, we developed a system with a Graphical User Interface (GUI) as depicted by Figure 2. In this system, the user can generate a grid with sizes of 10x10, 20x20, or 30x30. There, Long Term Features (LTFs) are static blocked cells, colored in black, where the robot can not traverse, like walls in a real indoor environment. The system randomly creates theses cells depending on the parameter %LTF.

The user can set the start (green cell) and goal (red cell) states. At any time, the user can stop the system and add Short Term Features (STFs) and Dynamic Features (DFs). STFs are represented by purple cells which also block the path of the robot, but they are not part of the static map as LFTs. They can be seen as furniture or any other movable object. DFs are orange cells which dynamically change their positions over time, like people and other moving objects. After selecting one of the available algorithms, the system computes a path and starts the execution.

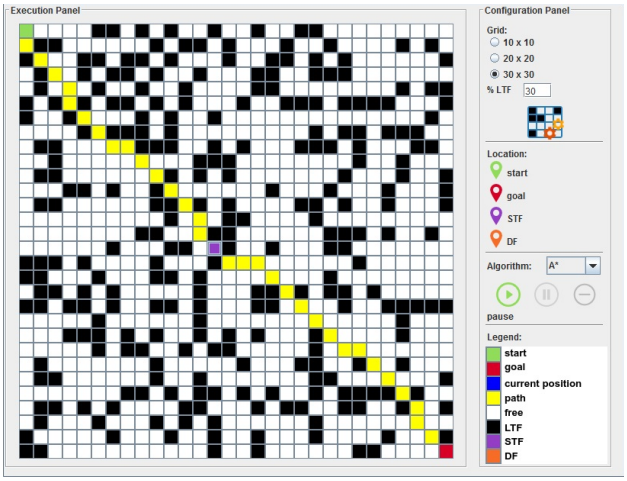

Fig. 2: Graphical User Interface (GUI)

We generated some scenarios using grids of 10x10, 20x20, and 30x30 cells. Two search algorithms were tested, *D*Lite with Human Assistance* and *A**. The tests were carried out by computing paths that cross the grid. After planning, the execution of the path begins. Then, some objects were added on the middle of the path to force the replanning. *D*Lite with Human Assistance* tries to replan the path efficiently while *A** plans another path from scratch.

Usually, search algorithms have an queue, called *OPEN* list, that maintains states to be explored during the search. Thus, to analyse the performance of both algorithms, we tracked how many states were stored into the *OPEN* lists while the algorithms were replanning.

Figure 3 depicts the results of both approaches. The horizontal axis of the charts displays the number of states in the *OPEN* list and the vertical axis shows the iteration number during the search. As can be seen, the number of states explored using *D*Lite with Human Assistance* is much lower than the number of states used by *A**. Also, it requires fewer iterations to perform the search.

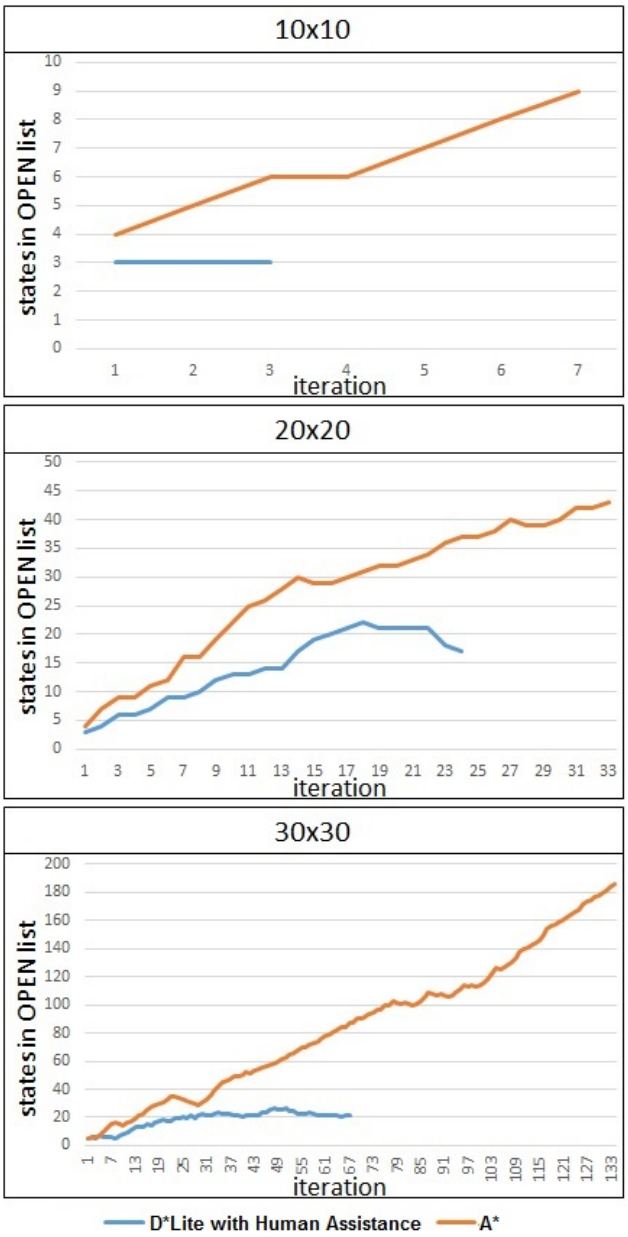

Fig. 3: Replanning: OPEN list

*A** algorithm needs to explore as many states as necessary to find the optimal path, which requires more CPU and memory resources to evaluate the states stored in the *OPEN* list. This

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on January 08,2026 at 12:48:25 UTC from IEEE Xplore. Restrictions apply.

can be a problem for big and complex environments. Therefore, in the next experiment, we evaluate the performance of a relaxed version of *A** againts *D*Lite with Human Assistance*. This *Relaxed A** does not track all possible paths to the goal state but only the best, working almost as a *Best First Search*.

We implemented both algorithms on *Robot Operating System* (ROS) as been Global Path Planners. ROS is an open-source, meta-operating system for robots. It provides services expect from an operating system, including hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management. It also provides tools and libraries for obtaining, building, writing, and running code across multiple robots and computers. With ROS, we can run our algorithms on real and virtual robots in the same way.

For this test, we used a virtual *Turtlebot* on *Gazebo*. *Gazebo* is a 3D simulator that has an interface with ROS. This simulater allows the creation of 3D scenarios with robots, obstacles and many other objects. It also uses a physical engine for illumination, gravity, inertia, etc. To track the internal state of the robot we used ROS visualization (RViz) which is a 3D visualizer for displaying sensor data and state information from the robot. In this experiment we evaluated the quality of the paths produced by *D*Lite with Human Assistance* and *Relaxed A**. We generated four different paths on the indoor virtual environment depicted by Figure 4.

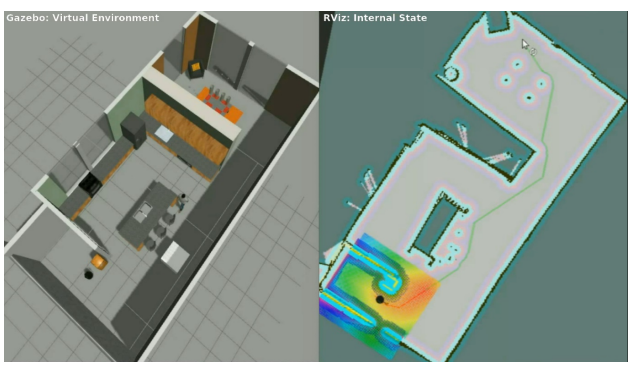

Fig. 4: Simulation with Turtlebot, ROS, Gazebo, and RViz

The charts in Figure 5 show that our approach yields shorter paths for longer distances or at least paths of the same length for smaller distances.

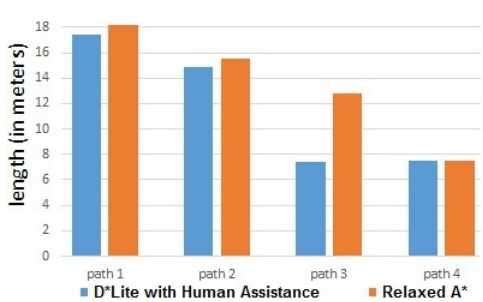

Fig. 5: Path length

These two experiments show that we kept the properties of *D*Lite* regarding the efficiency of building paths in dynamic environments. For the last experiment, we investigated the impact of human assitance during the navigation in simple and complex environments using virtual and real scenarios, like those illustrated by Figures 6 7.

For the virtual scenarios we used a virtual *Turtlebot* on *Gazebo*, and for the real ones we used our physical *Turtlebot* with the same ROS codes that we developed.

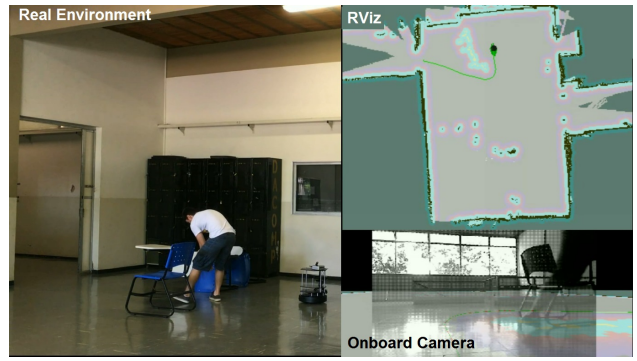

Fig. 6: Corridor: 2nd floor of Building-B at Federal University of Uberlandia

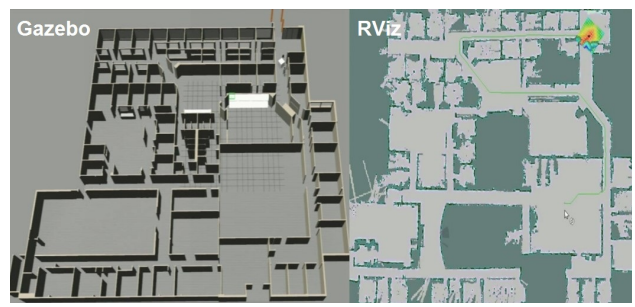

Fig. 7: Willow Garage Inc: virtual environment of approximately 53m x 42m

In this experiment we also added some obstacles further away in the middle of the the robot's path. When the robot detects theses objects it replans its path. Then we calculated the difference between the original and the replanned path as been the deviation necessary to the robot reaches the destination.

TABLE I: Navigation Time

Environment	Deviation	D*Lite		D*Lite with H. A.	
		Waiting	Navigation	Waiting	Navigation
Simple	3.97m	0s	**1:00**	5s	**0:58**
Complex	43.82m	0s	**6:00**	10s	**2:50**

Table I presents the navigation time for a simple scenario and a complex scenario using *D*Lite* with and without human assistance. The first column shows the type of the environment. The second column shows the deviation length. Then we show

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on January 08,2026 at 12:48:25 UTC from IEEE Xplore. Restrictions apply.

the results for *D*Lite*. The column *Waiting* shows the time that the robot waited for help, in this case is 0 as the robot always decide to take the replanned path. The column *Navigation* shows the total navigation time to reach the destination since the beginning, which were 1:00 minute for the simple scenario and 6:00 minutes for the complex scenario. Finally, we present the results for *D*Lite with Human Assistance*. The robot waited 5 seconds in the simple scenario and 10 seconds in the complex scenario to a human move the blocking objects away. Thus, the total navigation time for the simple and complex scenarios were respectively 0:58 minutes and 2:50 minutes.

As we can see, for small deviations the navigation time is almost the same. However, for long deviations the human assistance can decrease the navigation time and eventually save robot's resources such as battery.

V. Conclusion

Service Robots are typically autonomous mobile robots that can operate in indoor areas in order to help people in their daily tasks. However, many of these tasks require the navigation in changing environments.

*D*Lite* is a dynamic path planner which replans more efficiently in case the robot's path is obstructed by objects presented in the environment. In this paper, we proposed an adaptation to the *D*Lite* that allows the robot to ask for human assistance in a symbiotic manner.

Experiments show that our approach maintains *D*Lite* properties and also accelerates the navigation by controlling whether and how long the robot can wait for a human to move objects away in its surroundings before it takes a path.

VI. Acknowledgements

This research is supported in part by the National Council for Scientific and Technological Development of Brazil (CNPq) and Federal University of Uberlandia (UFU).

References

[1] Z. Teresa, "History of service robots," in *Concepts, Methodologies, Tools, and Applications*. IGI Global, 2014, pp. 1–14. [Online]. Available: http://dx.doi.org/10.4018/978-1-4666-4607-0.ch001

[2] N. J. N. P. E. Hart and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems, Science, and Cybernetics*, vol. SSC-4, no. 2, pp. 100–107, 1968.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *NUMERISCHE MATHEMATIK*, vol. 1, no. 1, pp. 269–271, 1959.

[4] A. Stentz, "Optimal and efficient path planning for partially-known environments," in *Robotics and Automation, 1994. Proceedings., 1994 IEEE International Conference on*. IEEE, 1994, pp. 3310–3317.

[5] S. Koenig and M. Likhachev, "D*lite," in *Eighteenth National Conference on Artificial Intelligence*. Menlo Park, CA, USA: American Association for Artificial Intelligence, 2002, pp. 476–483. [Online]. Available: http://dl.acm.org/citation.cfm?id=777092.777167

[6] D. T. Wooden, "Graph-based path planning for mobile robots," Ph.D. dissertation, Georgia Institute of Technology, 2006.

[7] M. Stilman and J. J. Kuffner, "Navigation among movable obstacles: Real-time reasoning in complex environments," *International Journal of Humanoid Robotics*, vol. 2, no. 04, pp. 479–503, 2005.

[8] S. Koenig and M. Likhachev, "Incremental a*." in *NIPS*, 2001, pp. 1539–1546.

[9] A. Stentz and M. Hebert, "A complete navigation system for goal acquisition in unknown environments," *Autonomous Robots*, vol. 2, pp. 127–145, 1995.

[10] M. Hebert, R. MacLachlan, and P. Chang, "Experiments with driving modes for urban robots," in *Photonics East'99*. International Society for Optics and Photonics, 1999, pp. 140–149.

[11] L. Matthies, Y. Xiong, R. Hogg, D. Zhu, A. Rankin, B. Kennedy, M. Hebert, R. Maclachlan, C. Won, T. Frost *et al.*, "A portable, autonomous, urban reconnaissance robot," *Robotics and Autonomous Systems*, vol. 40, no. 2, pp. 163–172, 2002.

[12] D. Ferguson, M. Likhachev, and A. Stentz, "A guide to heuristic-based path planning," in *Proceedings of the international workshop on planning under uncertainty for autonomous systems, ICAPS*, 2005, pp. 9–18.

[13] D. Ferguson and A. Stentz, "The delayed d* algorithm for efficient path replanning," in *Proceedings of the 2005 IEEE international conference on robotics and automation*. IEEE, 2005, pp. 2045–2050.

[14] D. Cockburn and Z. Kobti, "Memory-bounded d* lite." in *FLAIRS Conference*, D. Wilson and H. C. Lane, Eds. AAAI Press, 2008, pp. 376–380. [Online]. Available: http://dblp.uni-trier.de/db/conf/flairs/flairs2008.html#CockburnK08

[15] G. A. Mills-Tettey, A. Stentz, and M. B. Dias, "Dd* lite: Efficient incremental search with state dominance." in *Proceedings of the National Conference on Artificial Intelligence*, vol. 21, no. 2. Menlo Park, CA; Cambridge, MA; London; AAAI Press; MIT Press; 1999, 2006, p. 1032.

[16] D. Ferguson and A. Stentz, "Field d*: An interpolation-based path planner and replanner," in *Robotics research*. Springer, 2007, pp. 239–253.

[17] R. Philippsen, "A light formulation of the e* interpolated path replanner," 2006. [Online]. Available: https://doi.org/10.3929/ethz-a-010113621

[18] M. M. Veloso, J. Biswas, B. Coltin, and S. Rosenthal, "Cobots: Robust symbiotic autonomous mobile service robots." in *IJCAI*, Q. Yang and M. Wooldridge, Eds. AAAI Press, 2015, pp. 4423–. [Online]. Available: http://dblp.uni-trier.de/db/conf/ijcai/ijcai2015.html#VelosoBCR15

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on January 08,2026 at 12:48:25 UTC from IEEE Xplore. Restrictions apply.